

Unit-I

Basics of Python

- Python is an interpreted, object-oriented programming language.
- It was created by Guido van Rossum in 1989, and released in 1991.
- ABC programming language is said to be the predecessor of Python language.
- Python is influenced by following programming languages:

*ABC language

*Modula-3

-This programming language was named as Python because. One comedy Series named- Monty Python's Flying Circus.

Applications of Python:

Python is known for its general-purpose nature that makes it applicable in almost every domain of software development.

Here, we are specifying application areas where Python can be applied.

1) Web Applications:

We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML, XML, Email processing ,etc.

One of Python web-framework named Django is used on Instagram.

2) Desktop GUI Applications:

The GUI stands for the Graphical User Interface, which provides a smooth interaction to any application. Python provides a **Tk GUI library** to develop a user interface.

3) Console-based Application:

Console-based applications run from the command-line or shell. These applications are computer program which are used commands to execute. Ex. text editor

4) Image Processing Application:

Python contains many libraries that are used to work with the image. The image can be manipulated according to our requirements. Ex. Face Detection , Background Removal

5) Software Development:

Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

6) Scientific and Numeric:

This is the era of Artificial intelligence where the machine can perform the task the same as the human. Python language is the most suitable language for Artificial

intelligence or machine learning. It consists of many scientific and mathematical libraries, which makes easy to solve complex calculations. Ex. SciPy , Pandas

***Python Features/ Characteristics :**

1) Easy to Learn and Use -

Python is easy to learn and use. It is developer-friendly and high level programming language.

2) Expressive Language -

Python language is more expressive means that it is more understandable and readable.

3) Interpreted Language -

Python is an interpreted language i.e. interpreter executes the code line by line at a time. This makes debugging easy and thus suitable for beginners.

4) Cross-platform Language -

Python can run equally on different platforms such as Windows, Linux, Unix and Macintosh etc. So, we can say that Python is a portable language.

5) Free and Open Source -

Python language is freely available at official web address. The source-code is also available. Therefore it is open source.

6) Object-Oriented Language -

Python supports object oriented concepts.

7) Large Standard Library -

Python has a large and broad library and provides rich set of module and functions for rapid application development.

8) GUI Programming Support -

Graphical user interfaces can be developed using Python.

9) Integrated -

It can be easily integrated with languages like C, C++, JAVA etc.

10) Dynamically Typed -

Many programming languages need to declare the type of the variable before runtime. With Python, the type of the variable can be decided during runtime. This makes Python a dynamically typed language. Ex. `x=15`

*Python Virtual Machine(PVM):



-Computers understands only machine code i.e. 1s and 0s. Our source code is in high level language which needs to convert in machine level language.

-Compiler normally converts the source code into machine code called as byte code size of each byte code (instruction) is 1 byte.

-Byte code is platform independent. The roll of Python Virtual Machine (PVM) is to convert byte code into machine code. Computer can execute this machine code and gives final output.

-The PVM is equipped with an interpreter. Interpreter checks and understand processor and OS. according to this interpreter converts byte code into machine code and sends that machine code to processor for execution.

-Since interpreter plays main role, often the PVM is also called as interpreter

Memory Management in Python

Python handles memory management automatically, making it easier for developers to focus on coding rather than worrying about memory allocation and deallocation.

1. Automatic Memory Allocation

- Python uses a private heap to store objects and data structures.

- When you create variables or objects, Python automatically allocates memory for them.

2. Garbage Collection

- Python has a built-in garbage collector that identifies and removes unused objects to free up memory.
- Garbage Collection in Python is an automatic process that handles memory allocation and deallocation, ensuring efficient use of memory.
- Garbage Collection is a process of automatically freeing objects.

3. Memory Pools

- Python uses a memory manager to handle small objects efficiently. It organizes memory into pools to reduce fragmentation and improve performance.

4. del Keyword

- You can use the del keyword to delete references to an object manually, but this doesn't guarantee immediate memory release. The garbage collector will handle it when appropriate.

Install of Python on Windows :-

Downloading Python :-

First things first, let's get Python onto your computer. Head over to the official Python website (python.org) and look for the Downloads section. You'll see different versions available, but for beginners, I recommend sticking with the latest stable version of Python 3.

Installation of Python:-

To install Python on a Windows machine, follow these steps:

- Download Python for Windows.
- Run the Python installer you downloaded.

- Click "Install Now"

- Here, you will see the Python wizard, which is very easy to use. Just accept the default recommended settings and click on the Next button, wait until the installation is complete, and you are done.



Python - Environment Setup :-

Setting path at Windows

Windows users who checked "Add Python to PATH" during installation can skip this step. If you didn't, no worries! Here's what to do:

1. Right-click on 'This PC' and choose 'Properties'
2. Click on 'Advanced system settings'
3. Click on 'Environment Variables'
4. Under 'System variables', find and select 'Path', then click 'Edit'
5. Click 'New' and add the path to your Python installation (usually something like C:\Python39.....)

writing the first Python program:-

write a Hello World program in Python -

Python program contains code statements created in a text file and saved with the extension `py`.

- Open any editor Ex. IDLE
- Create a file called HelloWorld.py, where `py` is an extension for Python code statements code.
- Add the below code

```
print("Hello world Welcome to Python programming")
```

- Execute – Click on Run -> Run Module or Press F5 button

Executing a Python program :-

How to compile and execute the Python program -

To execute and run a Python code, a Python compiler is used to compile and execute a program.

The compiler compiles program code to bytecode or native code and

- `Python` tool compiles the Python to native code and executes the code

The `python` tool compiles the Python code into native code.

1)open Editor(Notepad) -> Create and Save Python Program(HelloWorld.py)

2)Open CMD -> compile and run python program

```
Py HelloWorld.py
```

It generates and prints the string

```
Hello world Welcome to Python programming.
```

Data Types In Python:-

Data types are the classification or categorization of data items.

Python is a dynamically typed language so we need not to define data type of a variable while declaring it.
type() is a Built-In function used to know data type of any variable.

Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

Data Types Can be 1) Built-In

2) User Defined .

Let's see 1) Built-In Data types :-

There are various Built-In data types in Python.

Numeric Types: **int, float, complex**

Text Type: **string**

Sequence Types: **list, tuple**

Mapping Type: **Dict (Dictionary)**

Set Types: **Set**

Boolean Type: **Bool**

Binary Types: **bytes**

1) Numeric Type:

The numeric data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, or even a complex number

- There are three numeric types in Python:
 - int**: for integral values
 - float**: for float value
 - complex**: for complex numbers
- Variables of numeric types are created when we assign a value to them.

```
x = 1 # int
y = 2.8 # float
z = 1j # complex

print(type(x))
print(type(y))
print(type(z))

#type() function is to know data type of any variable
```

Here we assigned numeric values to variables x,y, and z and we used **type()** function to know data types of variables x,y, and z.

Type Conversion:

Type Conversion is a process to convert one data type value in another.

We can convert from one type to another with the **int()**, **float()**, and **complex()** methods:

Example:

```
x = 1 # int
y = 2.8 # float

#convert from int to float:
a = float(x)

#convert from float to int:

b = int(y)

print(type(a))
print(type(b))
```

1)Implicit Type Conversion in Python :-

In Implicit type conversion of data types in Python, the Python interpreter automatically converts one data type to another without any user involvement.

Ex.

```
x = 10
print("x is of type:",type(x))
y = 10.6
print("y is of type:",type(y))
z = x + y
print(z)
print("z is of type:",type(z))
```

Output

```
x is of type: <class 'int'>
y is of type: <class 'float'>
20.6
z is of type: <class 'float'>
```

2)Explicit Type Conversion

In Explicit Type Conversion in Python, the data type is manually changed by the user as per their requirement.

Ex.

```
x = 1 # int
y = 2.8 # float
#convert from int to float:
a = float(x)
#convert from float to int:
b = int(y)
```

```
print(type(a))
print(type(b))
```

O/P :-

```
<class 'float'>
```

```
<class 'int'>
```

2) String:

String is collection or group of characters , string is a sequence of characters, representing textual data .String literals in python are surrounded by either single quotation marks, or double quotation marks or Triple quotation.

Ex. 'Hello' or "Hello" or """Hello"""

```
a = "Hello"
print(a)
```

3) List:

A list is a collection of elements or items which is ordered and changeable.

List allows duplicate members.

In Python lists are written with square brackets. One list can contain another list as Element.

Example to create a List:

```
x = [1,2,3,"apple",[11,22]]
print(x)
```

- We will see more about List in next chapter.

4) Tuple:

- A tuple is a collection of elements which is ordered and unchangeable.
- Tuple allows duplicate members.
- In Python tuples are written with round brackets.
- A Tuple can contain another tuple.

Example to create a Tuple:

```
y = ("Sangola",1,55,3)
print(y)
```

5) Set:

- **Set** is a collection of elements which is unordered and indexed.
- There are no duplicate members in Set.
- In Python set is written with curly brackets.
- Example to create a Set:

```
z = {5,10, "cherry"}
print(z)
```

6) Dictionary:

- A dictionary is a collection which is unordered, changeable and indexed. Dictionary do not allows duplicate members.
- In Python dictionaries are written with curly brackets, and they have keys and values.
- Example

```
a = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 2019  
}  
  
print(a)
```

Here, “brand”, ”model” and ”year” are the keys.

“ford”, ”Mustang” and ”2019” are the values

We will see more about Dictionary in next chapter.

7) Boolean Type:

- Booleans represent one of two values: True or False.
- In programming we often need to know if an expression is True or False.
- We can evaluate any expression in Python, and get one of two answers, True or False.
- When we compare two values, the expression is evaluated and Python returns the Boolean answer
- The bool() function allows us to evaluate any value, and give us True or False in return

```
print(10 > 9)
```

```
print(bool("Hello"))  
  
print(bool(0))
```

8) Bytes Type:

- The **bytes** type in Python is immutable and stores a sequence of values ranging from 0-255 (8-bits). bytes() method in Python is used to create a sequence of bytes.

```
y=bytes(4)  
  
print(y)  
  
print(type(y))
```


Python Comments :-

Comments can be used to explain Python code. Comments can be used to make the code more readable.

Creating a Comment :-

Comments starts with a # :-

Ex.1)Single Line comments:-

```
#This is a comment.  
print("Hello, World!")
```

2) `print("Hello, World!") #This is a comment`

3) Multi-Line comments:-

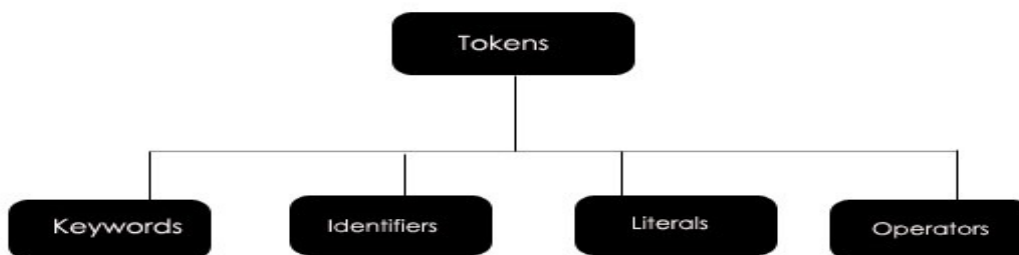
```
"""This is a comment  
written in  
more than just one line"""  
print("Hello, World!")
```

Tokens In Python:

“Token is nothing but smallest individual unit of Python program.”

Python program has classes and every classes has some methods and methods contains executable statement and every executable statement contains the tokens i.e. statements are made up of several tokens. Python supports 4 types of Tokens

1) Keywords 2) Identifiers 3) Literals 4) Operators



1) Keywords :-

The Keywords are reserved words, whose meaning is already known by Python compiler. Each keyword have unique and fixed meaning and we are not able to change that meaning therefore they are also called as 'Reserved Words'

Let's see Python Keywords

<u>False</u>	<u>await</u>	<u>else</u>	<u>import</u>	<u>pass</u>
<u>None</u>	<u>break</u>	<u>except</u>	<u>in</u>	<u>raise</u>
<u>True</u>	<u>class</u>	<u>finally</u>	<u>is</u>	<u>return</u>
<u>and</u>	<u>continue</u>	<u>for</u>	<u>lambda</u>	<u>try</u>
<u>as</u>	<u>def</u>	<u>from</u>	<u>nonlocal</u>	<u>while</u>
<u>assert</u>	<u>del</u>	<u>global</u>	<u>not</u>	<u>with</u>
<u>async</u>	<u>elif</u>	<u>if</u>	<u>or</u>	<u>yield</u>

2) Python Identifiers :-

“Identifier is the name given by the programmer for any variable, package, class, interface, array, object etc.”

It helps to differentiate one entity from another.

Rules for writing identifiers

i) Identifiers can be a combination of letters in lowercase or uppercase or digits or an underscore `_`.

ex. `myClass`, `var_1` and `print_this_to_screen`, all are valid example.

ii) An identifier cannot start with a digit.

ex. `1name` (It is invalid)

iii) Keywords cannot be used as identifiers.

ex. `global = 1` (It is invalid because `global` is a keyword)

iv) We cannot use special symbols like `!`, `@`, `#`, `$`, `%` etc. in our identifier.

v) An identifier can be of any length.

3) Literals :-

A literal represent a fixed value that is stored into variable directly in the program. They are represented directly in the code without any computation.

Literals can be assigned to any variable.

e.g `gstr="welcome"` or `x=10` etc.

following are types of literals,

- String literals :- `"hello"`, `'12345'`
- Int literals :- `0`, `1`, `2`, `-1`, `-2`
- Long literal- :: `89675L`
- Float literals :- `3.14`
- Complex literals :- `12j`
- Boolean literals :- `True` or `False`
- Special literals :- `None`
- Unicode literals :- `u"hello"`
- List literals :- `[]`, `[5,6,7]`
- Tuple literals :- `()`, `(9,)`, `(8,9,0)`
- Dict literals :- `{}`, `{'x':1}`
- Set literals :- `{8,9,10}`

4) Operators :-

Operators are special symbols in Python that operates on operands. And the operands are nothing but Data values.

Python divides the operators in the following groups:

i) Arithmetic operators

ii) Assignment operators

iii) Comparison operators

iv) Logical operators

v) Identity operators (Special Operator)

vi) Membership operators (Special Operator)

vii) Bitwise operators

i) Arithmetic operators :-

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operator	Meaning	Example
+	Add two operands or unary plus	$x + y + 2$
-	Subtract right operand from the left or unary minus	$x - y - 2$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

***Program for Arithmetic operators:-**

```
x=15
y=5
p=x+y
print("\n Addition=",p)
q=x-y
print("\n subtraction=",q)
```

ii) Assignment operators :-

Assignment operators are used in Python to assign values to variables.

The left side of assignment operator always contains a variable where right side can be value, variable or an expression.

***Program for Assignments operators:-**

```
x=15
print("\n Assignment operator:",x)
x+=5      #same like x=x+5
print("\n Addition assignment:",x)
x/=5      #same like x=x/5
print("\n Division assignment:",x)
```

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5

iii) Comparison operators :-

Comparison operators are used to compare values.

It returns either True or False according to the condition.

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y
<	Less than - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

***Program for comparison operator :-**

```
x=15
y=20
if(x>y):
print("\n X is greater")
else:
print("\n Y is greater")
```

iv) Logical operators :-

Logical operators are used to compare two or more conditional statements.

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

-Here logical and operator returns true result if and only if both conditions are true.

-The logical or operator returns true result when both the conditions are true

Also returns true value when at least one condition is true

- The logical not operator complements the result .i.e. True converted in false
And false converted in true.

* Program for logical **and** operator :-

```
x=15
y=20
if(x>y and x==15):
    print("\n Condition is true")
else:
    print("\n Condition is false")
```

Output: Condition is false

* Program for logical **or** operator :-

```
x=15
y=20
if(x>y or x==15):
    print("\n Condition is true")
else:
    print("\n Condition is false")
```

Output: Condition is True

*Program for logical **and** operator :-

```
x=15
y=20
if not(x>y):
    print("\n Condition is true")
else:
    print("\n Condition is false")
```

Output: Condition is True

v) Identity Operators :-

is and **is not** are the identity operators in Python. They are used to check if two values (or variables) are located on the same part of the memory. Two variables that are equal does not imply that they are identical.

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	x is True
is not	True if the operands are not identical (do not refer to the same object)	x is not True

Program for is operator

```
x=20
y=20
if (x is y):
    print("x and y have same memory location")
else:
    print("x and y have different memory location")
```

output: x and y have same memory location

In above program variable x and y have same value i.e. 20, we know that if multiple variables have same values then a common memory location is allocated to those same valued variables.
is operator returns **True** when multiple variables have common memory location.

Program for is not operator

```
x=10
y=20
if (x is not y):
    print("x and y have different memory location")
else:
    print("x and y have Same memory location")
```

output: x and y have different memory location

is not operator returns **True** when x and y have **different** memory location.
is not operator returns **False** when x and y have **same** memory location.

vi) Membership operators :-

in and **not in** are the membership operators in Python. They are used to test whether a value or variable is found in a sequence (string, list, tuple, set and dictionary).
In a dictionary we can only test for presence of key, not the value.

Operator	Meaning	Example
in	True if value/variable is found in the sequence	'H' in x
not in	True if value/variable is not found in the sequence	'H' not in x

Program for in operator

```
x = 'Hello world'
print('H' in x)
Output: True
```

Program for not in operator

```
x = 'Hello world'
print('z' not in x)
Output: True
```

vii) Bitwise operators :-

Bitwise operators act on operands as if they were strings of binary digits. They operate bit by bit, hence the name is bitwise operators.

OR

Python bitwise operators are used to perform bitwise calculations on integers. The integers are converted into binary format and then operations are performed bit by bit, hence the name bitwise operators

Following are bitwise operators

Note: Refer Truth table for Bit operation

In the table below: Let `x = 10` (`0000 1010` in binary) and `y = 4` (`0000 0100` in binary)

Operator	Meaning	Example
<code>&</code>	Bitwise AND	<code>x & y = 0</code> (<code>0000 0000</code>)
<code> </code>	Bitwise OR	<code>x y = 14</code> (<code>0000 1110</code>)
<code>~</code>	Bitwise NOT	<code>~x = -11</code> (<code>1111 0101</code>)
<code>^</code>	Bitwise XOR	<code>x ^ y = 14</code> (<code>0000 1110</code>)
<code>>></code>	Bitwise right shift	<code>x >> 2 = 2</code> (<code>0000 0010</code>)
<code><<</code>	Bitwise left shift	<code>x << 2 = 40</code> (<code>0010 1000</code>)

Program for Bitwise operators :-

```
x=10
y=4
print("\n output of bitwise and=",int(x&y));
print("\n output of bitwise or=",int(x|y));
print("\n output of bitwise not=",int(~x));
print("\n output of bitwise EXOR=",int(x^y));
print("\n output of bitwise shift right=",int(x>>2));
print("\n output of bitwise shift Left=",int(x<<2));
```

Output :-

output of bitwise and = 0

output of bitwise or = 14

output of bitwise not = -11

output of bitwise EXOR = 14

output of bitwise shift right = 2

output of bitwise shift Left = 40

*** Operator Precedence:**

The operator precedence in Python is listed in the following table. It is in descending order (upper group has higher precedence than the lower ones). The operator precedence defines the order in which operators are evaluated.

Let's see operator precedence,

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisons, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Ex. `print(5 * 2 - 3)`

*** Associativity of Python Operators :-**

- We can see in the previous table that more than one operator exists in the same group. These operators have the same precedence.
- When two operators have the same precedence, associativity helps to determine the order of operations.
- Associativity is the order in which an expression is evaluated that has multiple operators of the same precedence. Almost all the operators have left-to-right associativity.
- For example, multiplication and floor division have the same precedence. Hence, if both of them are present in an expression, the left one is evaluated first.

- **# Left-right associativity**

```
print(5 * 2 // 3)
```

```
# Output: 3
```



```
# Shows left-right associativity
print(5 * (2 // 3))
# Output: 0
```

- Note:- Exponent operator ** has right-to-left associativity in Python.
Shows the right-left associativity of **

```
print(2 ** 3 ** 2)
```

```
# Output: 512
Since , 2**(3**2) = 2**9
```

- # If 2 needs to be exponentated first, need to use ()

```
print((2 ** 3) ** 2)
```

```
# Output: 64
```

*** Non associative operators :-**

Some operators like assignment operators and comparison operators do not have associativity in Python. There are separate rules for sequences of this kind of operator and cannot be expressed as associativity.

Naming conventions :-

Naming conventions in Python refer to rules and guidelines for naming variables, functions, classes, and other entities in your code.

Here, we discuss the Naming Conventions in Python which are follows.

- Modules
- Variables

1) Modules :-

Modules in Python are files containing Python definitions and statements. When naming a module, use lowercase letters and underscores. Let's create a module named **math_operations.py**

Ex.

```
def add_numbers(a, b):
```

```
    result = a + b
```

```
    print("The sum is ",result)
```

```
add_result = add_numbers(5, 3)
```

O/p :- The sum is: 8

2) Variables :-

Globals variable should be in uppercase with underscores separating words, while locals variable should follow the same convention as functions.

Ex.

```
GLOBAL_VARIABLE = 10
```

```
def example_function():
```

```
    local_variable = 5
```

```
    print(GLOBAL_VARIABLE + local_variable)
```

```
# Call the function to print the result
```

```
example_function()
```

Variable:-“Variable is the name given to the memory location where the data is stored”.

Unlike other programming languages, Python has no command for declaring a variable.

A variable is created the moment we first assign a value to it.

- There are several rules to declare the variable:
 - 1) Variable should not be keyword.
 - 2) Variable should not start with digit.
 - 3) Variable can be combination of alphabets, digits or underscore.
 - 4) Variable should not contain special symbol except underscore.
 - 5) Variable should not contain any white space character like horizontal tab, vertical tab, new line etc.
 - 6) Variable can be of any length.

Example.

```
x = 5
y = "Sangola"
print(x)
print(y)
```

Here,

Variables do not need to be declared with any particular type and can even change type after they have been set.

String variables can be declared either by using single or double quotes:

print() is a function which is used to print anything in python

There are two types of variables in python

- 1) Local Variable
- 2) Global variable

1)Local Variable:

The variables which are declared inside methods, constructors or blocks are called local variables.

These variables are declared and initialized within the method and they will be destroyed automatically when the method completes its execution.

2)Global variable

The variables which are declared outside methods, constructors or blocks are called global variables.

The scope of global variable is entire program.

Assigning multiple values to variables:

Python allows us to assign multiple values to multiple variables in one line.

```
x,y,z="Python" ,"Java" ,"C++"  
print(x)  
print(y)  
print(z)
```

Constants:

Constant is a value that remains the same and does not change or it cannot be modified.

Python does not allow us to create constants as easy as we do in C, C++ or Java.

In Python, we can define constants in a module, where a module is a file which could be imported to access the defined constants and methods.

Note: A module constant should be in uppercase alphabets, where multi-word constant

Example:

Step 1 : Create a module and define a constant.

save module file by any name

Here we save module as **conex.py**

```
# Defining constants in a module/file
```

```
INK = 'RED'
```

```
PI = 3.14
```

Step 2: import our module in python terminal or in new script and use it.

To access our constant following syntax will be used

Modulename.constatname

```
import conex  
  
print('Value of constant APPLE : ',conex.INK)  
  
print('Value of constant GRAVITY : ',conex.PI)
```

Here first we imported our module as import conex after that we accessed our constants INK and PI as conex.INK and conex.PI

Input and Output statements :

- In every programming language there is a need to get input from user and provide the output to user
- Here In python we can get input from user and provide the output for that some functions which are as follows

Python Output Using print() function

Python Input using input() function

- These two are built-in functions

1)print() function :

- We use the `print()` function to output data to the standard output device (screen).
- We can also output data to a file, but this will be discussed later.
- An example of its use is given below.

```
print('This sentence is output to the screen')
```

- Another example is given below:

```
a = 5
print('The value of a is', a)
```

2)input() function :-

- Up until now, our programs were static. The value of variables was defined or hard coded into the source code.
- To allow flexibility, we might want to take the input from the user. In Python, we have the `input()` function to allow this.
- The syntax for `input()` is:

`input([prompt])`

- where ,prompt is the string we wish to display on the screen. It is optional.

`s=input('enter any string')`

Note: `input()` function accepts all inputs in string format so we have to do typecasting as per necessary.

Command Line Arguments :-

The Argument that is passing at the time of execution is called the **Command Line Argument**.

Using sys.argv :-

The sys module provides functions and variables used to manipulate different parts of the Python runtime environment and maintained by the interpreter. One such variable is `sys.argv`, which is a simple list structure.

Its main purpose is:

- It is a list of command line arguments.
- `len(sys.argv)` provides the number of command line arguments.
- `sys.argv[0]` is the name of the current Python script.

Eg: `D:\Python_classes py test.py 10 20 30`


Command Line Arguments

Within the Python Program, these Command Line Arguments are available in `argv`. Which is present in `SYS` Module.

<code>test.py</code>	<code>10</code>	<code>20</code>	<code>30</code>
----------------------	-----------------	-----------------	-----------------

Note: `argv[0]` represents Name of Program. ,`argv[1]` represents the First Command Line Argument.

To display Command Line Arguments :-

Example :-

```
from sys import argv
print("The Number of Command Line Arguments:", len(argv))
print("The List of Command Line Arguments:", argv)
print("Command Line Arguments one by one:")
for x in argv:
    print(x)
```

Execute on Command Prompt

```
C:\Users\SATISH\Desktop>py test.py 10 20 30
The Number of Command Line Arguments: 4
The List of Command Line Arguments: ['test.py', '10', '20', '30']
Command Line Arguments one by one:
test.py 10 20 30
```

```
from sys import argv
sum=0
args=argv[1:]
for x in args :
    n=int(x)
    sum=sum+n
print("The Sum:",sum)
```

O/P:

```
D:\Python_pro>
py test.py 10 20 30 40
The Sum: 100
```

`argv` → a list of command-line arguments passed to the Python script.

- `argv[0]` → the name of the script itself (e.g., "myscript.py").
- `argv[1:]` → all the arguments after the script name.
- The slice `[1:]` takes the list from index 1 up to the end, skipping index 0.
- `args` → is now a list that stores only the user-provided command-line arguments, excluding the script name.
- It comes from the `sys` module:

*Control Statements/Structure:

The control structure is used to control flow of execution of python program.

The control structure are as follows

- a) If...else statements (Decision making)
- b) Looping (for loop and while loop)
- c) break and continue statements

a) Decision Making :

- Decision making is required when we want to execute a code only if a certain condition is satisfied.

- We can make decision with the help of,

if Statement

if...else statement

if...elif...else Statement

Nested if statements

Let's see them in brief

i) if statement:

- Python gives us a conditional **if statement**, which is used when we want to test a condition.
- This *condition* is evaluated for a boolean value **true** or **false**.

Syntax

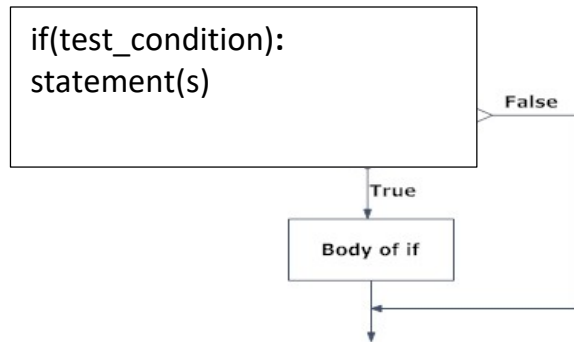


Fig: Operation of if statement

Example for if:

```
x=10
y=5
if(x>y):
    print("x is greater")
```

Output: X is greater

ii) if..else statement:

- The if..else statement evaluates test condition and will execute the body of if only when the test condition is True.
- If the condition is False, the body of else is executed.
- Indentation is used to separate the blocks.

Syntax of if...else

```
if(test_condition):
    Body of if
else:
    Body of else
```

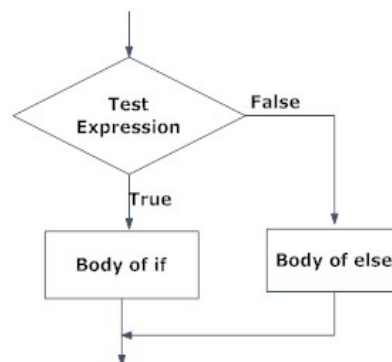


Fig: Operation of if...else statement

Example for if..else

```
x=10
y=5
if(x>y):
    print("x is greater")
else:
    print("y is greater")
```

Output: X is greater

iii) if..elif..else statement :-

- The **elif** is short for else if. It allows us to check for multiple expressions.

- If the condition for if is False, it checks the condition of the next elif block and so on.
- If all the conditions are False, the body of else is executed.

Syntax of if...elif...else

```
if(test_condition):
    Body of if
elif(test_condition):
    Body of elif
else:
    Body of else
```

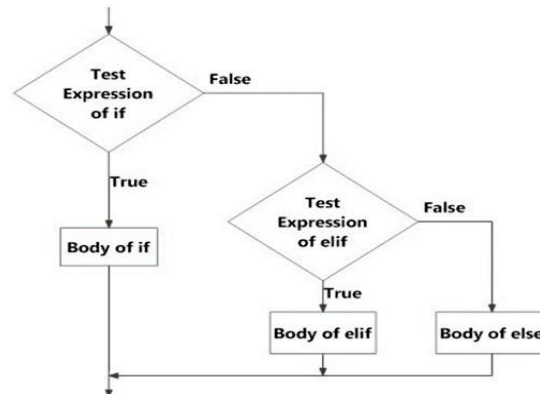


Fig: Operation of if...elif...else statement

Example for if..elif..else

```
per=55
if(per>70):
    print("Distinction")

elif(per>60 and per<=70):
    print("First class")

elif(per>50 and per<=60):
    print("Second class")
```

Output: Second class

iv) Nested if statements:

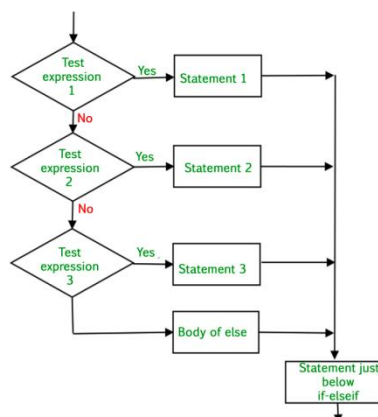
An if statement inside if statement is called as Nested if statement.

Any number of these statements can be nested inside one another.

Indentation is the only way to figure out the level of nesting. They can get confusing, so they must be avoided unless necessary.

Syntax for nested if

```
if(test_condition):
    body
    if(test_condition):
        body
```



Example for nested if

```
x=120
if(x<200):
    print("value is less than 200")
if(x<150):
    print("\n also less than 150")
```

Output -value is less than 200
also less than 150

b) Looping(Loops):

A Loop is sequence of instruction/s that repeated until certain condition is reached.

There are following loops present in python

i) **for loop**

ii) **while loop**

i) **for loop:**

- The for loop in Python is used to iterate over a sequence ([list](#), [tuple](#), [string](#)) or other iterable objects.
- Iterating over a sequence is called traversal.

Syntax of for Loop

```
For variable name in sequence:  
    Body or logic
```

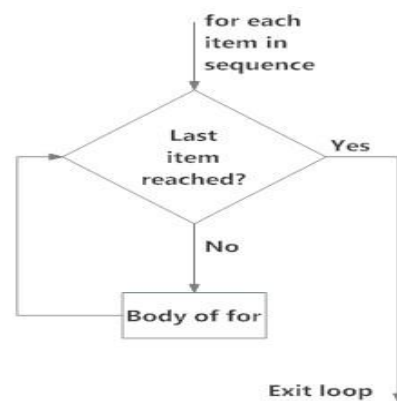


Fig: operation of for loop

Here each item from sequence is assigned to variable one by one. The loop terminates when there is no any item left in sequence.

Program of for loop

```
x=[0,1,5]
for p in x:
    print(p)
```

In our program **x** is a List(sequence) which have three Elements.

By using **for** loop each element of list is assigned to variable **p** after that these elements printed by function **print()**

for loop with else :

- A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts.
- The break keyword can be used to stop a for loop. In such cases, the else part is ignored.
- Hence, a for loop's else part runs if no break occurs.

Example

```
x=[0,1,5]
for p in x:
    print(p)
    #break
else:
    print("No items left")
```


for loop range() function:

We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).

We can also define the start, stop and step size **as range(start, stop-1, step_size)**.
step_size defaults to 1 if not provided

Program

```
x=[0,1,5]
for p in range(1,10):
    print(p)
```

We used x as,

Range(1,10) where 1 is start and 10 is stop. By this a sequence if 1 to (10-1)=9 is created.

The step size is optional part

ii) while loop : The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.

- We generally use this loop when we don't know the number of times to iterate before hand.

Syntax of while Loop in Python

```
while (test_expression):
    Body of while
```

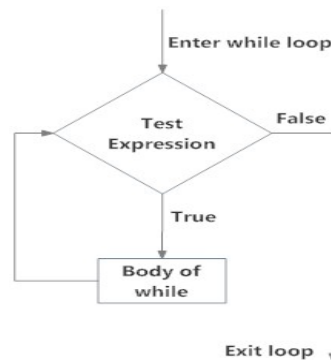


Fig: operation of while loop

Program to print 1 to 10 using while loop

```
i=1
while(i<=10):
    print(i)
    i+=1
```

While loop with else:

- Same as with [for loops](#), while loops can also have an optional else block.
- The else part is executed if the condition in the while loop evaluates to False.
- The while loop can be terminated with a [break statement](#).
- In such cases, the else part is ignored.
- Hence, a while loop's else part runs if no break occurs and the condition is false.

```
i=1
while(i<=10):
    print(i)
    i+=1
    break
else:
    print("else executed")
```

Output: 1

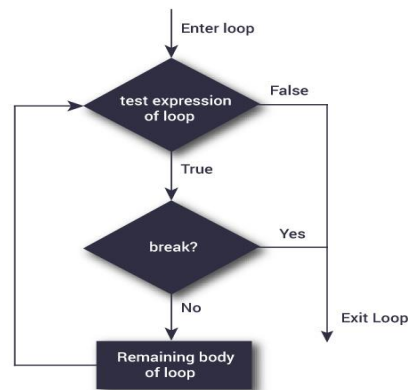
Our while loop iterates only one time
Because we used break statement.
Also our else part don't get executed
Because of break statement

c) break and continue:

- In Python, break and continue statements can alter the flow of a normal loop.
- Loops iterate over a block of code until the test expression is false, but sometimes we wish to terminate the current iteration or even the whole loop without checking test expression.
- The break and continue statements are used in these cases.

* break:

- The break statement terminates the loop containing it.
- Control of the program flows to the statement immediately after the body of the loop.
- If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.



Program for break

```
for x in range(1,5):  
    print(x)  
    break
```

Output- 1

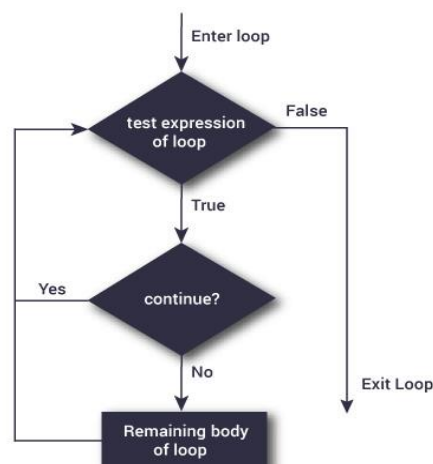
In our program for loop iterates only once
Because of break statement hence output is 1

* continue :

- The continue statement is used to skip the rest of the code inside a loop for the current iteration only.
- Loop does not terminate but continues on with the next iteration.

Program for continue

```
for val in "string":  
    if val == "i":  
        continue  
    print(val,end="")  
print("\nThe end")
```



Python pass statement :

- In Python programming, the pass statement is a null statement.
- The difference between a comment and a pass statement in Python is that while the interpreter ignores a comment entirely, pass is not ignored.

However, nothing happens when the pass is executed. It results in no operation (NOP).

- The pass statement is used as a placeholder for future code.
- When the pass statement is executed, nothing happens, but you avoid getting an error when empty code is not allowed.
- Empty code is not allowed in loops, function definitions, class definitions, or in if statements.

```
s = {'p', 'a', 's', 's'}
for val in s:
    pass
```

Assert keyword :

- The assert keyword is used when debugging code.
- The assert keyword lets we test if a condition in our code returns True, if not, the program will raise an AssertionError.

```
a=10
b=0
assert b!=0
print (a/b)
```

```
def sample(): #function definition
print ("Hello world") #function body
sample() #function calling
```

The return statement:

The return statement is used to exit a function and go back to the place from where it was called also it returns some value. In python we can return multiple values from function.

Syntax of return:-

```
return [expression_list]
```

Ex.

```
def sub(p,q):
    r=p-q
    return r
m=sub(15,5)
print("\n Subtraction is",m)
print("\n Subtraction is=",sub(25,10))
```

word indentation :-

word indentation mean spacing at the beginning of a line

- ❑ **Indentation** = spaces or tabs at the **beginning of a line**.
- ❑ In Python, indentation is **not optional**.
- ❑ It is used to **define blocks of code** (loops, functions, conditionals, classes, etc.).

Ex.

if True:

```
    print("This is indented") # 4 spaces indentation
```

```
    print("Still inside the if block")
```

```
print("Outside the if block") # No indentation
```

Wrong Indentation (Error):

Ex.

if True:

```
print("This will give IndentationError") #wrong indentation
```

IndentationError: expected an indented block

Infinite loop :-

An **infinite loop** in Python is a loop that never ends (keeps running until the program is stopped manually).

Ex.1)

while True:

```
    print("This will run forever")
```

True is always true → so the loop never stops.

You can stop it manually by pressing Ctrl + C in the terminal.

Ex.2)

```
x = 1
```

```
while x > 0: # This condition is always true
```

```
    print("Infinite loop")
```

Ex.3)

Breaking an Infinite Loop

You can use break to exit when a condition is met:

while True:

```
    num = int(input("Enter a number (0 to stop): "))
```

```
    if num == 0:
```

```
        print("Loop stopped")
```

```
        break
```

```
    print("You entered:", num)
```

* **Nested Loop :**

A loop inside loop is called as nested loop.

Program for nested loop

```
rows=6
for i in range(rows):
    for j in range(i):
        print('*',end=" ")
    print(" ")
```

1)Armstrong Number :-

```
n=int(input("Enter no="))
sum=0
temp=n
while n>0:
    rem=n%10
    sum=sum+(rem*rem*rem)
    n=int(n/10)
if sum==temp:
    print("Number is armstrong")
else:
    print("Number is not armstrong ")
```

O/P:-

Enter no=153

Number is armstrong

2)Armstrong Range:-

```
for n in range(1,500):
    sum=0
    for i in range(1,n):
        temp=n
        while n>0:
            rem=n%10
            sum=sum+(rem*rem*rem)
            n=int(n/10)
        if sum==temp:
            print(temp)
```

O/P:-

153

370

371

407

3) Magic Number:-

```
n=int(input('enter no='))
sum=0
while n>9:
    while n!=0:
        rem=n%10
        sum=sum+rem
        n=int(n/10)
if sum==1 or sum==10:
    print('Magic Number')
else:
    print('not')
```

3) Magic Number Range:-

```
sum=0
for i in range(1,501):
    n=i
    while n>9:
        while n!=0:
            rem=n%10
            sum=sum+rem
            n=int(n/10)
        n=sum
        sum=0
    if(n==1):
        print(i)
```

4) Palindrome Number :-

```
n=int(input("Enter no="))
sum=0
temp=n
while n>0:
    rem=n%10
    sum=sum*10+rem
    n=int(n/10)
if sum==temp:
    print("Number is palindrome")
else:
    print("not")
```

5) Palindrome Range:-

```
for n in range(1,501):
    sum=0
    for i in range(1,n):
        temp=n
        while n>0:
            rem=n%10
            sum=sum*10+rem
            n=int(n/10)
        if sum==temp:
            print(temp)
```

6) Perfect Number :-

```
n=int(input("Enter no="))
sum=0
for i in range(1,n):
    if(n%i==0):
        sum=sum+i
if(sum==n):
    print("Number is perfect")
else:
    print("not perfect")
```

7) Perfect Range :-

```
for n in range(1,501):
    sum=0
    for i in range(1,n):
        if(n%i==0):
            sum=sum+i
    if n==sum:
        print(n)
```

8) Prime Number :-

```
n=int(input("enter any no"))
i=2
while i<=n-1:
    if n%i==0:
        break
    i+=1
if i==n:
    print("no is Prime")
else:
    print("not prime")
```

9)Prime Range :-

```
for n in range(1,501):  
    i=2  
    while i<=n-1:  
        if n%i==0:  
            break  
        i+=1  
    if i==n:  
        print(n)
```